

Video Diffusion Models

Complete Technical Analysis

For From-Scratch Implementation

Paper Reference

Ho, Salimans, Gritsenko, Chan, Norouzi, Fleet
“Video Diffusion Models” - arXiv:2204.03458, April 2022
Google Research, Brain Team

Table of Contents

1	Introduction	3
1.1	Paper Overview	3
1.2	Key Contributions	3
1.3	Challenge	3
2	Diffusion Model Background	4
2.1	Forward Process	4
2.2	Log-SNR	4
2.3	Training Objective	4
2.4	Classifier-Free Guidance	4
3	3D U-Net Architecture	5
3.1	Overview	5
3.2	Space-Time Factorization	5
3.3	Block Structure	5
3.4	Spatial Attention	5
3.5	Temporal Attention	5
3.6	Architecture Configuration	6
4	Training Methodology	7
4.1	Cosine Schedule	7
4.2	Joint Image-Video Training	7
4.3	Training Configuration	7
4.4	Training Loop	7
5	Sampling & Reconstruction Guidance	8
5.1	Ancestral Sampling	8
5.2	DDIM Sampling	8
5.3	Reconstruction Guidance	8
5.4	Applications	8
6	Experiments	9
6.1	UCF101 Results	9
6.2	Video Prediction	9
6.3	Reconstruction Guidance Ablation	9
7	Implementation Guide	10
7.1	Architecture Mapping	10
7.2	Noise Schedule	10
7.3	DDIM Sampling	10
7.4	Implementation Checklist	11
8	References	12

1 Introduction

1.1 Paper Overview

Paper Information

Title: Video Diffusion Models

Authors: Ho, Salimans, Gritsenko, Chan, Norouzi, Fleet

Affiliation: Google Research

ArXiv: 2204.03458 (April 2022)

Significance: First video diffusion model

1.2 Key Contributions

Five Major Contributions

1. **3D U-Net Architecture:** Extended image diffusion to video
2. **Space-Time Factorized Attention:** $1000\times$ speedup
3. **Joint Image-Video Training:** $3.4\times$ FVD improvement
4. **Reconstruction Guidance:** Novel conditional generation
5. **State-of-the-Art Results:** Best on UCF101, BAIR, Kinetics

1.3 Challenge

Memory Requirements

A 16-frame video at 64×64 :

$$\text{Data} = 16 \times 64 \times 64 \times 3 = 196,608 \text{ values} \quad (1)$$

$16\times$ more than a single image!

2 Diffusion Model Background

2.1 Forward Process

Definition 2.1.1 (Forward Diffusion):

$$q(\mathbf{z}_t \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_t; \alpha_t \mathbf{x}, \sigma_t^2 \mathbf{I}) \quad (2)$$

where α_t decreases and σ_t increases with t .

Reparametrization

$$\mathbf{z}_t = \alpha_t \mathbf{x} + \sigma_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}) \quad (3)$$

2.2 Log-SNR

Definition 2.2.1 (Log Signal-to-Noise Ratio):

$$\lambda_t = \log \left(\frac{\alpha_t^2}{\sigma_t^2} \right) \quad (4)$$

Decreases monotonically from $+\infty$ to $-\infty$.

2.3 Training Objective

Definition 2.3.1 (ε -Prediction Loss):

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}, \varepsilon, t} [\|\varepsilon_{\theta(\mathbf{z}_t, t)} - \varepsilon\|_2^2] \quad (5)$$

2.4 Classifier-Free Guidance

Definition 2.4.1 (CFG Formula):

$$\tilde{\varepsilon}_\theta = (1 + w)\varepsilon_{\theta(\mathbf{z}_t, \mathbf{c})} - w\varepsilon_{\theta(\mathbf{z}_t, \emptyset)} \quad (6)$$

3 3D U-Net Architecture

3.1 Overview

Model I/O

Input: Noisy video $z_t \in \mathbb{R}^{T \times H \times W \times C}$, timestep t

Output: Predicted noise $\varepsilon_\theta \in \mathbb{R}^{T \times H \times W \times C}$

3.2 Space-Time Factorization

Computational Breakthrough

Full 3D attention: $O(T^2 H^2 W^2)$

Factorized: $O(T H^2 W^2 + H W T^2)$

For $T = 16, H = W = 64$: $1000\times$ speedup!

3.3 Block Structure

Each block contains:

1. Spatial Convolution ($1 \times 3 \times 3$)
2. Spatial Attention over ($H \times W$)
3. Temporal Attention over T
4. Residual Connection

3.4 Spatial Attention

PyTorch: Spatial Attention

```
class SpatialAttn(nn.Module):
    def forward(self, x): # (B, C, T, H, W)
        B, C, T, H, W = x.shape
        x = rearrange(x, 'b c t h w -> (b t) (h w) c')
        out = self.attention(x) # Attend over H*W
        return rearrange(out, '(b t) (h w) c -> b c t h w',
                        b=B, t=T, h=H, w=W)
```

3.5 Temporal Attention

PyTorch: Temporal Attention

```
class TemporalAttn(nn.Module):
    def forward(self, x): # (B, C, T, H, W)
        B, C, T, H, W = x.shape
        x = rearrange(x, 'b c t h w -> (b h w) t c')
        out = self.attention(x) + self.rel_pos_bias
        return rearrange(out, '(b h w) t c -> b c t h w',
                        b=B, h=H, w=W)
```

3.6 Architecture Configuration

Parameter	Value
Base Channels	256
Channel Mults	(1, 2, 4, 8)
ResBlocks/Stage	2
Attention Heads	8
Frames	16
Resolution	64×64

Figure 1: Default configuration

4 Training Methodology

4.1 Cosine Schedule

Definition 4.1.1 (Cosine Schedule):

$$\alpha_t^2 = \cos^2\left(\frac{t+s}{1+s} \cdot \frac{\pi}{2}\right) \quad (7)$$

where $s = 0.008$. Log-SNR clipped to $[-20, 20]$.

4.2 Joint Image-Video Training

Key Result

Adding 8 images per video batch:

- **FVD**: 205 $\rightarrow$ 61 (3.4 $\times$ improvement)
- **FID**: 37 $\rightarrow$ 15 (2.5 $\times$ improvement)

4.3 Training Configuration

Parameter	UCF101	BAIR	Kinetics
Learning Rate	3e-4	2e-4	2e-4
Batch Size	256	256	512
Steps	60K	660K	220K
Hardware	128 TPU	128 TPU	256 TPU

Figure 2: Training hyperparameters

4.4 Training Loop

PyTorch: Training Step

```
def train_step(model, videos, optimizer):
    t = torch.rand(B)
    alpha, sigma = schedule(t)
    eps = torch.randn_like(videos)
    z_t = alpha * videos + sigma * eps

    eps_pred = model(z_t, t)
    loss = F.mse_loss(eps_pred, eps)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss
```

5 Sampling & Reconstruction Guidance

5.1 Ancestral Sampling

Theorem 5.1.1 (Posterior Mean):

$$\tilde{\mu}_{s|t} = e^{\lambda_t - \lambda_s} \left(\frac{\alpha_s}{\alpha_t} \right) z_t + (1 - e^{\lambda_t - \lambda_s}) \alpha_s \hat{x}_\theta \quad (8)$$

5.2 DDIM Sampling

Definition 5.2.1 (DDIM Update):

$$z_s = \alpha_s \hat{x}_\theta + \sigma_s \left(\frac{z_t - \alpha_t \hat{x}_\theta}{\sigma_t} \right) \quad (9)$$

Deterministic, 50-100 steps sufficient.

5.3 Reconstruction Guidance

Theorem 5.3.1 (Reconstruction Guidance): For $p(\mathbf{x}^b \mid \mathbf{x}^a)$:

$$\tilde{\mathbf{x}}_\theta^b = \hat{\mathbf{x}}_\theta^b - \frac{w_r \alpha_t}{2} \nabla_{z_t^b} \|\mathbf{x}^a - \hat{\mathbf{x}}_\theta^a\|_2^2 \quad (10)$$

Why Needed?

Naive replacement gives $\mathbb{E}[\mathbf{x}^b \mid \mathbf{z}_t]$, not $\mathbb{E}[\mathbf{x}^b \mid \mathbf{z}_t, \mathbf{x}^a]$. Reconstruction guidance adds the missing conditioning gradient!

5.4 Applications

1. **Video Prediction:** Generate future from past frames
2. **Autoregressive Extension:** Generate 64+ frame videos
3. **Temporal Super-Resolution:** Double frame rate
4. **Spatial Super-Resolution:** 64→128 pixels

6 Experiments

6.1 UCF101 Results

Method	FID↓	IS↑
MoCoGAN	26998	12.42
TGAN-v2	3431	28.87
DVD-GAN	–	32.97
DIGAN	577	30.24
VDM	295	57.00
Real Data	–	60.20

Figure 3: UCF101: VDM IS approaches real data

6.2 Video Prediction

Dataset	Prev. SOTA	VDM
BAIR (FVD↓)	86.9	66.9
Kinetics (FVD↓)	25.4	16.2

Figure 4: Video prediction benchmarks

6.3 Reconstruction Guidance Ablation

Method	FVD↓	FID↓	IS↑
Replacement	451	26	7.0
Recon. Guidance	136	14	10.3

Figure 5: 3.3× FVD improvement with reconstruction guidance

7 Implementation Guide

7.1 Architecture Mapping

Paper Concept	Python Class
3D U-Net	UNet3D
Spatial Attention	SpatialAttention
Temporal Attention	TemporalAttention
Cosine Schedule	CosineSchedule
ϵ -Loss	train_step()

Figure 6: Paper to code mapping

7.2 Noise Schedule

PyTorch: Cosine Schedule

```
class CosineSchedule:
    def __init__(self, s=0.008):
        self.s = s

    def __call__(self, t):
        f = torch.cos((t + self.s) / (1 + self.s) * pi / 2) ** 2
        f0 = math.cos(self.s / (1 + self.s) * pi / 2) ** 2
        alpha = torch.sqrt(f / f0)
        sigma = torch.sqrt(1 - f / f0)
        return alpha, sigma
```

7.3 DDIM Sampling

PyTorch: DDIM Sampler

```
@torch.no_grad()
def ddim_sample(model, schedule, shape, num_steps=50):
    z_t = torch.randn(shape)
    timesteps = torch.linspace(1, 0, num_steps + 1)

    for i in range(num_steps):
        t, s = timesteps[i], timesteps[i + 1]
        alpha_t, sigma_t = schedule(t)
        alpha_s, sigma_s = schedule(s)

        eps_pred = model(z_t, t.expand(shape[0]))
        x_pred = (z_t - sigma_t * eps_pred) / alpha_t
        z_t = alpha_s * x_pred + sigma_s * eps_pred

    return z_t
```

7.4 Implementation Checklist

Components to Implement

1. `CosineSchedule` - Noise schedule with log-SNR clipping
2. `SpatialAttention` - Self-attention over $H \times W$
3. `TemporalAttention` - Self-attention over T with relative pos
4. `ResBlock3D` - Convolution + attention + residual
5. `UNet3D` - Encoder-decoder with skip connections
6. Training loop with EMA
7. DDIM sampler with CFG
8. Reconstruction guidance

8 References

1. Ho et al. “Video Diffusion Models.” arXiv:2204.03458, 2022.
2. Ho et al. “Denoising Diffusion Probabilistic Models.” NeurIPS 2020.
3. Nichol & Dhariwal. “Improved DDPM.” ICML 2021.
4. Song et al. “Denoising Diffusion Implicit Models.” ICLR 2021.
5. Ho & Salimans. “Classifier-Free Diffusion Guidance.” NeurIPS Workshop 2021.